```
def my_attr
  @my_attr
end

def my_attr=(value)
  @my_attr = value
end
```

Quiz Solution

Here's a solution:

```
def add_checked_attribute(klass, attribute)
  eval "
    class #{klass}
      def #{attribute}=(value)
        raise 'Invalid attribute' unless value
        @#{attribute} = value
      end

      def #{attribute}()
        @#{attribute}
      end
    end
  "
end
```

Here's the *String of Code (141)* that gets evaluated when you call add_checked_attribute(String, :my_attr):

```
class String
  def my_attr=(value)
    raise 'Invalid attribute' unless value
    @my_attr = value
  end

  def my_attr()
    @my_attr
  end
end
```

The String class is treated as an *Open Class (14)*, and it gets two new methods. These methods are almost identical to those that would be generated by attr_accessor, with an additional check that raises an exception if you call my_attr= with either nil or false.

"That was a good start," Bill says. "But remember our plan. We only used eval to pass the unit tests quickly; we don't want to stick with eval for the real solution. This takes us to step 2."